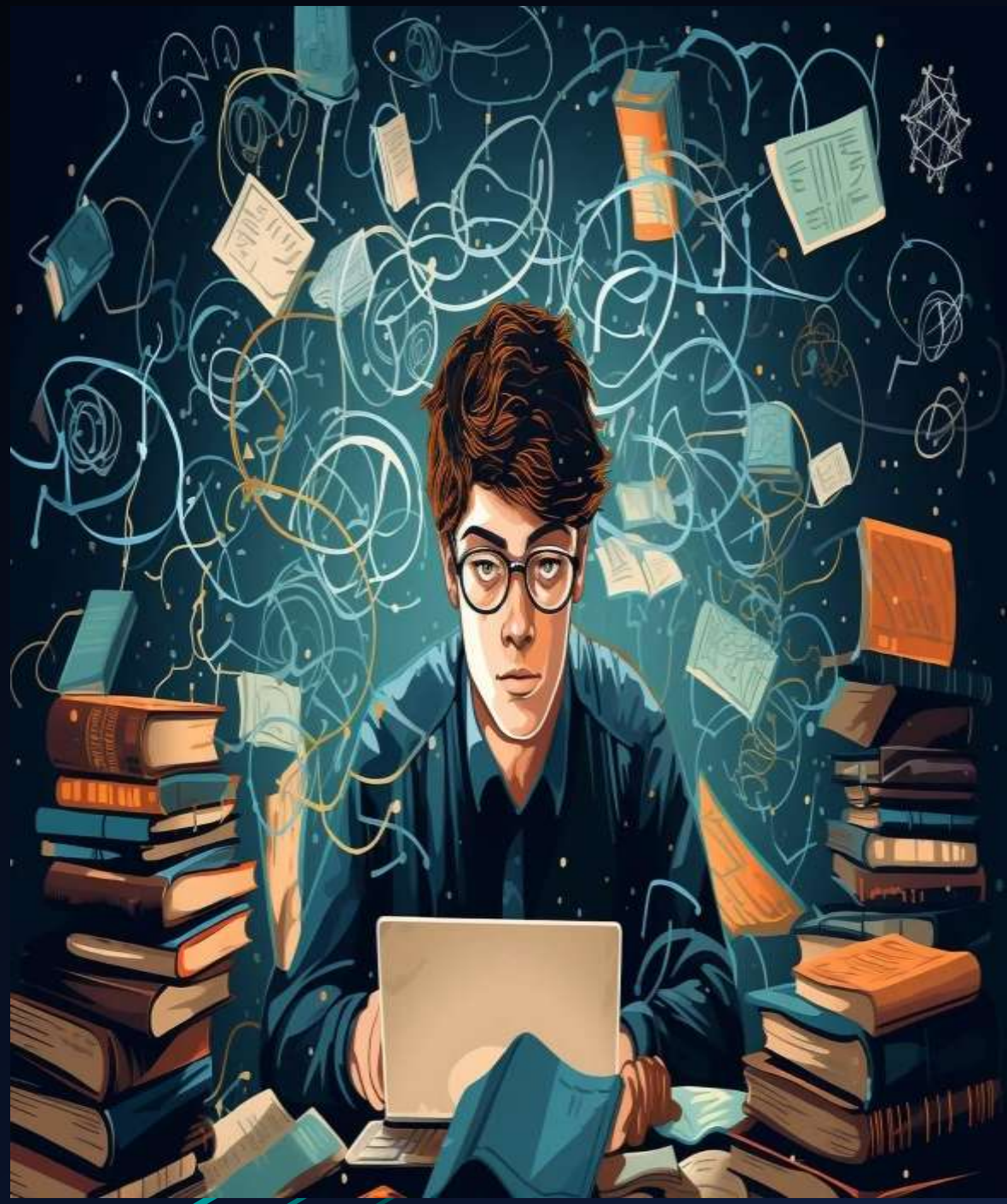
Student Performance Prediction Analysis

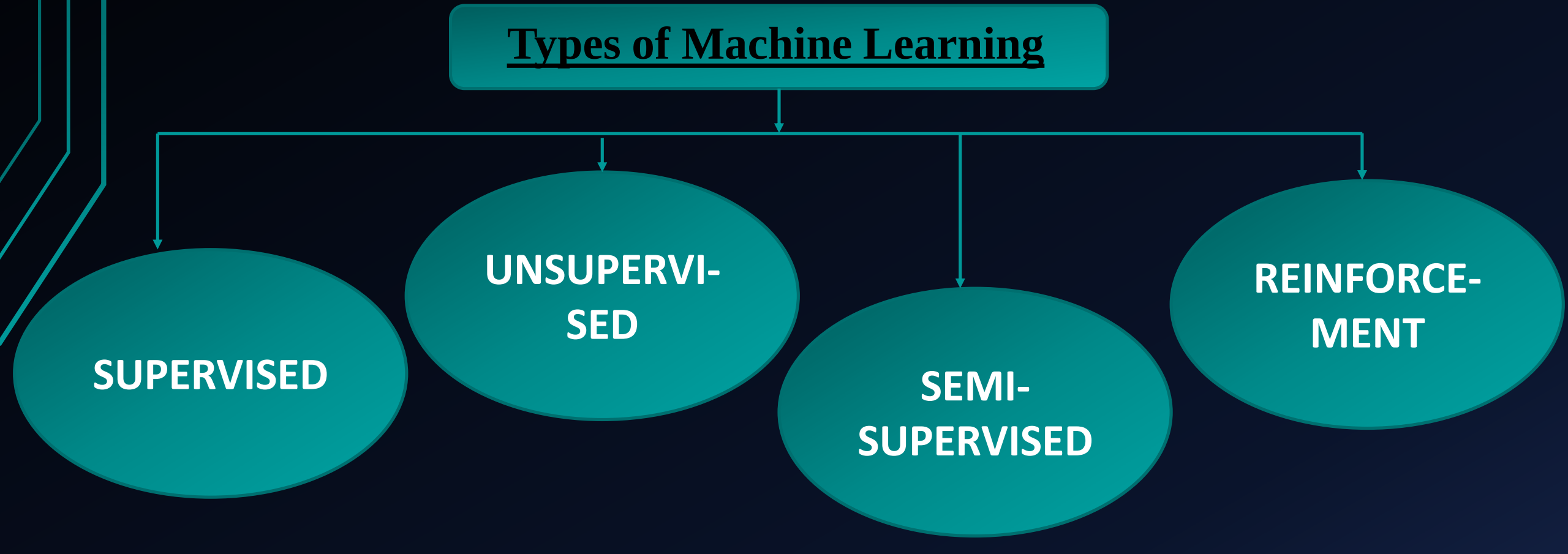
A DETAILED EXPLORATION OF STUDENT PERFORMANCE PREDICTION USING PYTHON AND MACHINE LEARNING.

AMAN ARORA



MACHINE LEARNING

Machine Learning (ML) enables computers to learn from data and make decisions without explicit programming, transforming industries through automation, insights, and innovation.



PURPOSE

- Assess the influence of study hours on academic performance.
- Examine the role of extracurricular activities in balancing academics.
- Evaluate the impact of previous scores on current performance trends.
- Analyze the effect of sleep hours on cognitive efficiency and outcomes.
- Measure the contribution of practicing sample question papers to performance improvement.
- Develop a comprehensive model to predict performance using daily activity data.
- Provide actionable insights for students to enhance their academic results.

DATASET

Loading the dataset is a crucial initial step in any data analysis process. It involves importing the data from a file or database into the working environment, ensuring it is structured and ready for processing. In Python, libraries like **Pandas** are commonly used to load datasets in formats such as CSV, Excel, or JSON. This step allows analysts to inspect the data, handle missing values, and prepare it for further analysis or modeling.

	Hours Studied	Previous score	Extracurricular Activities	Sleep Hours	Sample Question Papers Practiced	Performance Index
0	7	99	1	9	1	91
1	4	82	0	4	2	65
2	8	51	1	7	2	45
3	5	52	1	5	2	36
4	7	75	0	8	5	66
...	...	...	...	...	...	...
9995	1	49	1	4	2	23
9996	7	64	1	8	5	58
9997	6	83	1	8	5	74
9998	9	97	1	7	0	95
9999	7	74	0	8	1	64

10000 rows × 6 columns

DATA ABSTRACT

SHAPE OF DATASET
10000 ROWS*6 COLUMNS

HOURS STUDIED
PREVIOUS SCORES
EXTRACURRICULAR ACTIVITIES
SLEEP HOURS
SAMPLE QUESTION PAPERS PRACTICED
PERFORMANCE INDEX

MEAN OF DATASET
= 55.2248

MEDIAN OF DATASET
= 55.0

MODE OF DATASET
= 67

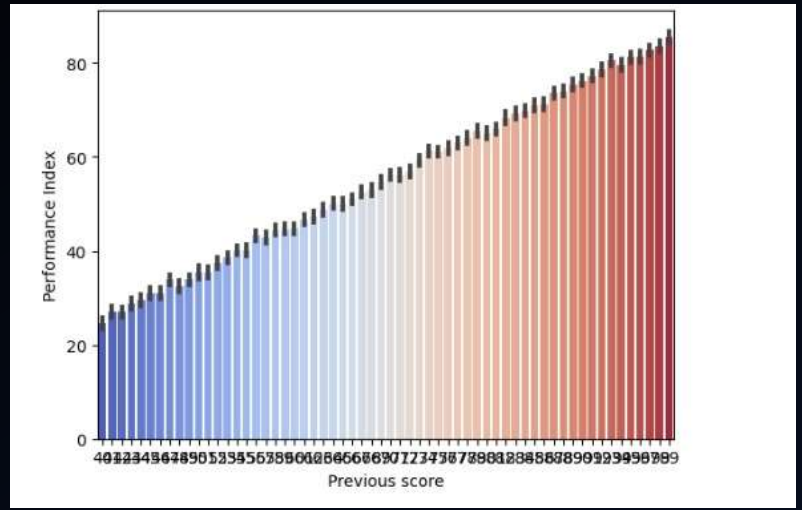
DESCRIPTION OF DATASET

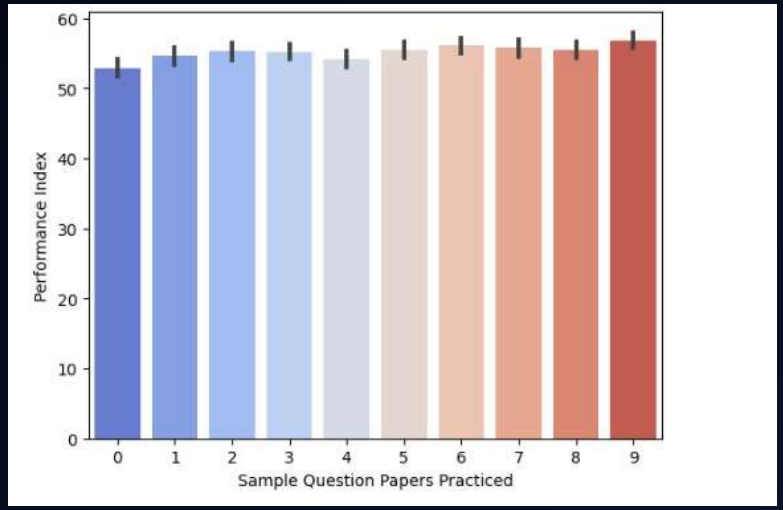
	Hours Studied	Previous score	Extracurricular Activities	Sleep Hours	Sample Question Papers Practiced	Performance Index
count	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000
mean	4.992900	69.445700	0.494800	6.530600	4.583300	55.224800
std	2.589309	17.343152	0.499998	1.695863	2.867348	19.212558
min	1.000000	40.000000	0.000000	4.000000	0.000000	10.000000
25%	3.000000	54.000000	0.000000	5.000000	2.000000	40.000000
50%	5.000000	69.000000	0.000000	7.000000	5.000000	55.000000
75%	7.000000	85.000000	1.000000	8.000000	7.000000	71.000000
max	9.000000	99.000000	1.000000	9.000000	9.000000	100.000000

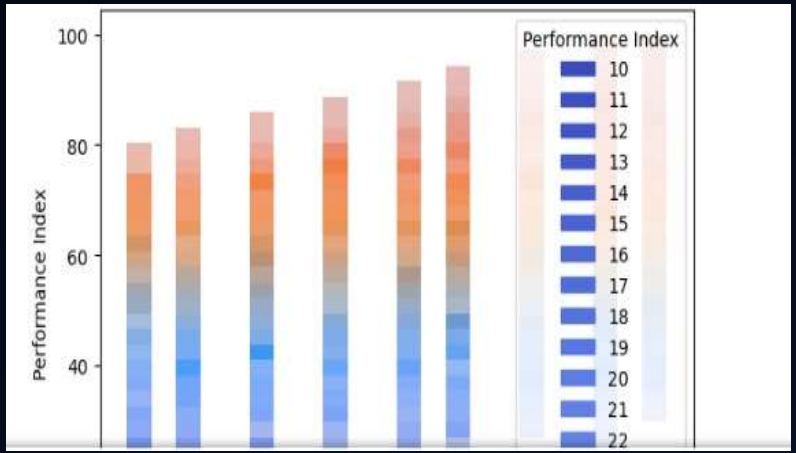
INFO OF DATASET

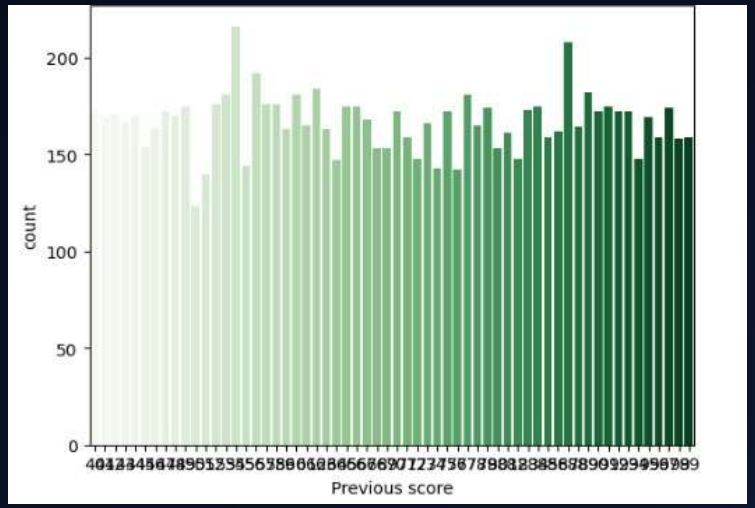
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 6 columns):
 #   Column                          Non-Null Count  Dtype
---  ------                          --------------  -----
 0   Hours Studied                   10000 non-null  int64
 1   Previous score                  10000 non-null  int64
 2   Extracurricular Activities      10000 non-null  int64
 3   Sleep Hours                     10000 non-null  int64
 4   Sample Question Papers Practiced  10000 non-null  int64
 5   Performance Index               10000 non-null  int64
dtypes: int64(6)
memory usage: 468.9 KB
```

GRAPH OF SOME PERFORMANCE



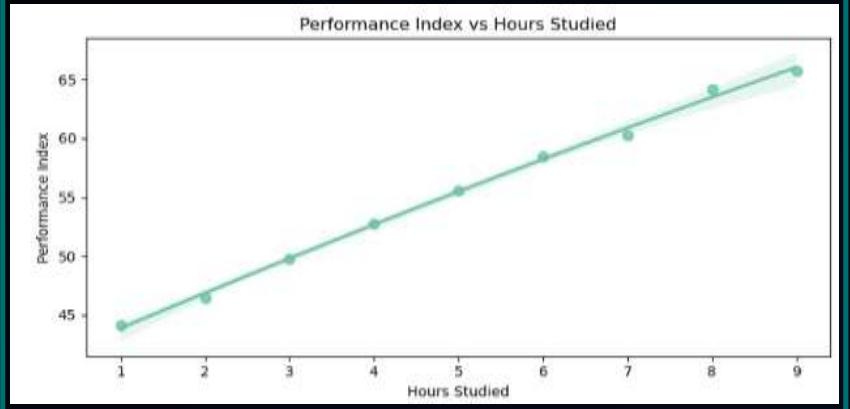


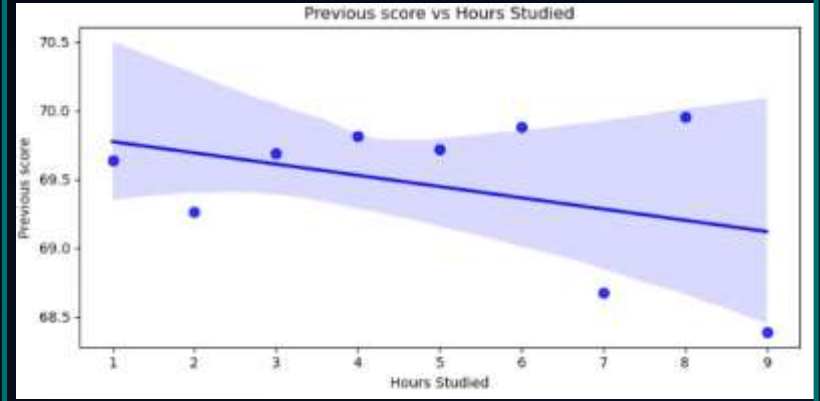


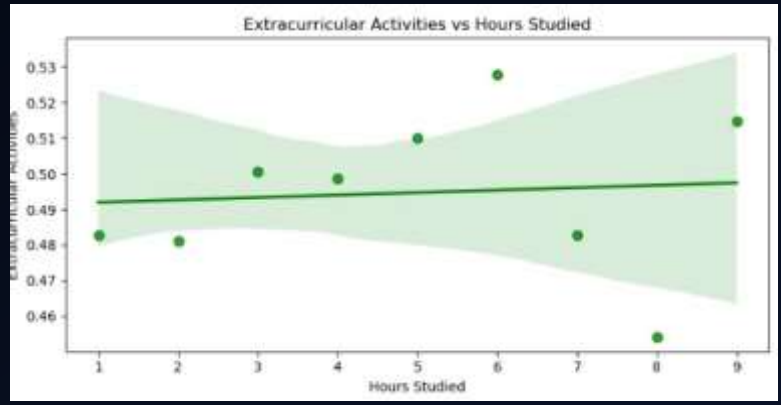


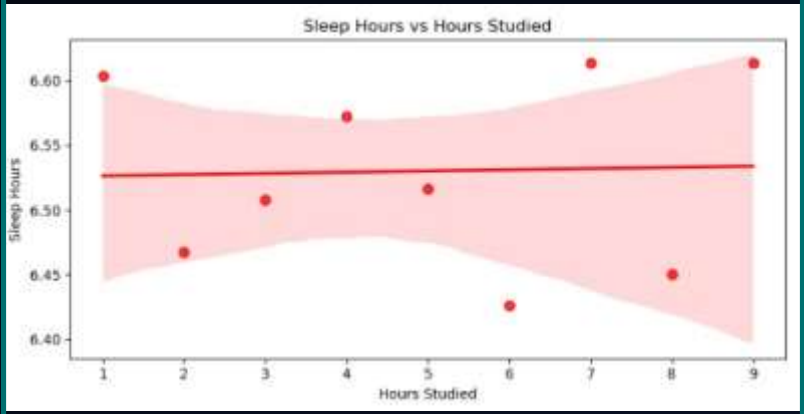
PERFORMANCE INDEX

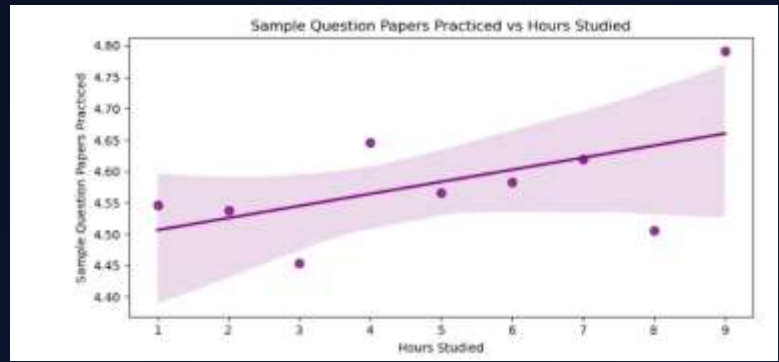
DIFFERENT FACTORS AFFECTING



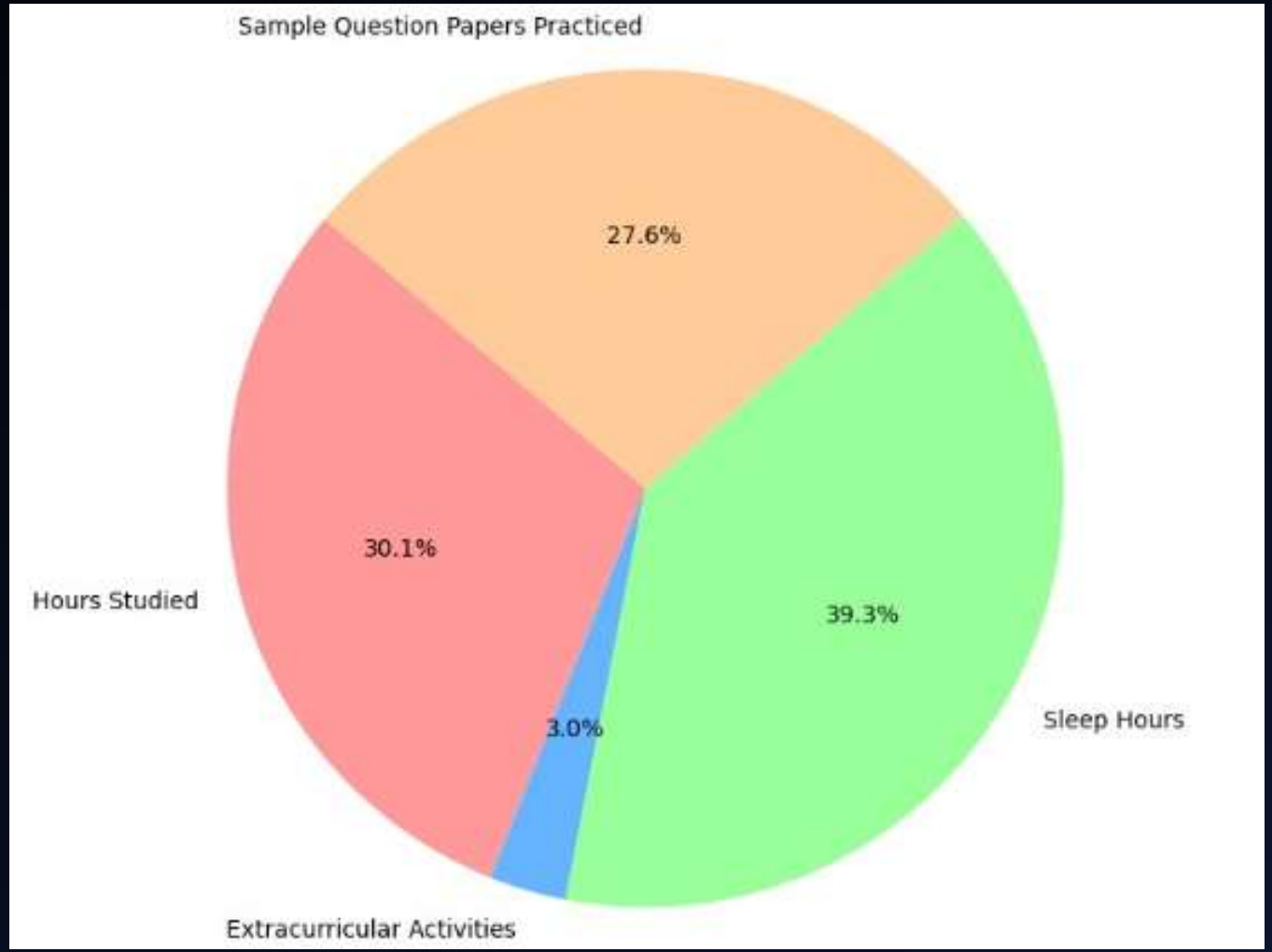








DISTRIBUTION OF FACTORS AFFECTING STUDENT PERFORMANCE



USING ALGORITHMS

LINEAR REGRESSION

Linear Regression is a fundamental statistical method used for modeling the relationship between a dependent variable (target) and one or more independent variables (predictors)

DECISION TREE

Random Forest Regressor is an ensemble algorithm that combines multiple decision trees to enhance prediction accuracy and reduce overfitting.

RANDOM FOREST REGRESSION

Decision Tree Regressor is a supervised algorithm that predicts continuous targets by splitting data into subsets based on feature thresholds.

TEST AND TRAIN SPLIT

```python
# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Display the shapes of the resulting datasets
print('X_train shape:', X_train.shape)
print('X_test shape:', X_test.shape)
print('y_train shape:', y_train.shape)
print('y_test shape:', y_test.shape)
```

	Hours Studied	Previous score	Extracurricular Activities	Sleep Hours	\
0	7	99	1	9	
1	4	82	0	4	
2	8	51	1	7	
3	5	52	1	5	
4	7	75	0	8	

	Sample Question Papers Practiced	Performance Index
0	1	91
1	2	65
2	2	45
3	2	36
4	5	66

```
X_train shape: (8000, 4)
X_test shape: (2000, 4)
y_train shape: (8000,)
y_test shape: (2000,)
```

DISPLAY TRAINING DATA

```
# Display the training data
print("Training Features (x_train):")
print(x_train.head())
print()
print("Training Target (y_train):")
print(y_train.head())
```

```
Training Target (y_train):
4901    33
4375    82
6698    74
9805    49
1101    43
Name: Performance Index, dtype: int64
```

Training Data Code Overview:

- **Import pandas:** The pandas library is imported for data manipulation.
- **Load Dataset:** The dataset is loaded from a CSV file into a pandas DataFrame using pd.read_csv().
- **Display Data:** The head() function is used to display the first few rows of the dataset, giving a quick preview of the data's structure and content. This process allows for efficient data inspection before further analysis or modeling.

SPLITING THE DATA

```
from sklearn.model_selection import train_test_split
# Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

This code splits the dataset into training and testing sets, with 80% of the data used for training and 20% for testing, ensuring reproducibility with a fixed random seed (random_state=42).

IMPLEMENTING LINEAR REGRESSOR

```
#Linear Regression
# building model
from sklearn.linear_model import LinearRegression
regressor = LinearRegression()
regressor
regressor.fit(X_train,y_train)
y_pred1 = regressor.predict(X_test)
y_pred1
# accuracy score
from sklearn.metrics import r2_score
score = r2_score(y_pred1,y_test)
print(score)
```

```
1.0
```

The code trains a *Linear Regression model* with X_train and y_train, predicts y_pred1 on X_test, and calculates the *R^2 score* for model accuracy using r2_score.

```
# Initialize and train the model
regressor = LinearRegression()
regressor.fit(X_train, y_train)  # Train the model on the training set
# Predicting on the test set
y_pred2 = regressor.predict(X_test)
# Calculating the R² score (accuracy)
score = r2_score(y_test, y_pred2)*100
print(f"R² score (Test Accuracy): {score:.4f}")
# Predicting for specific inputs (make sure the input dimensions match the training data)
Line1 = regressor.predict(np.array([[7, 99, 1, 9, 1]]))
Line2 = regressor.predict(np.array([[4, 82, 0, 4, 2]]))
Line3 = regressor.predict(np.array([[8, 51, 1, 7, 2]]))

print(Line1, Line2, Line3)
```

```
R² score (Test Accuracy): 100.0000
[1.] [2.] [2.]
```

The code trains a *Linear Regression model* on the training set (X_train, y_train), evaluates its accuracy using *R^2 score* on the test set (X_test, y_test), and predicts outcomes for specific inputs (Line1, Line2, Line3).

IMPLEMENTING RANDOM FOREST REGRESSOR

```
from sklearn.ensemble import RandomForestRegressor
dt1 = RandomForestRegressor()
dt1
dt1.fit(X_train,y_train)
y_pred3=dt1.predict(X_test)
y_pred3
from sklearn.metrics import r2_score
score2 = r2_score(y_pred3,y_test)*100
print(score)

Ran1 = regressor.predict([[7, 99, 1, 9, 1]])
Ran2 = regressor.predict([[4, 82, 0, 4, 2]])
Ran3 = regressor.predict([[8, 51, 1, 7, 2]])
print (Ran1,Ran2,Ran3)
```

```
100.0
[1.] [2.] [2.]
```

The code initializes a *Random Forest Regressor*, trains it on X_train and y_train, evaluates it using *R^2 score* on predictions (y_pred3) from X_test, and predicts outcomes (Ran1, Ran2, Ran3) for specific input arrays.

IMPLEMENTING DECISION TREE REGRESSOR

```
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import r2_score
dt1 = DecisionTreeRegressor()
dt1.fit(X_train, y_train)
y_pred3 = dt1.predict(X_test)
score3 = r2_score(y_test, y_pred3)*100
print(score3)

Dec1 = regressor.predict([[7, 99, 1, 9, 1]])
Dec2 = regressor.predict([[4, 82, 0, 4, 2]])
Dec3 = regressor.predict([[8, 51, 1, 7, 2]])
print (Dec1,Dec2,Dec3)
```

```
100.0
[1.] [2.] [2.]
```

The code initializes a *Decision Tree Regressor*, trains it with X_train and y_train, then evaluates the model's performance using *R^2 score* on the predictions (y_pred3) from X_test. It also makes predictions (Dec1, Dec2, Dec3) for specific inputs using the *Linear Regression model* (stored in regressor).

COMPARISON

```
import pandas as pd

# Creating the DataFrame with the given data
comparison_df = pd.DataFrame({
    "Algorithm": ["Linear Regressor", "Decision_Tree", "Random-Forest"],
    "Accuracy": [score,score2,score3],
    "Prediction-1": [Line1,Dec1,Ran1],
    "Prediction-2": [Line2,Dec2,Ran2],
    "Prediction-3": [Line3,Dec3,Ran3]})
# Display the DataFrame
comparison_df.head()
```

The code creates a DataFrame, comparison_df, to compare the results of three different algorithms (Linear Regressor, Decision Tree, and Random Forest). It includes columns for the algorithm names, their accuracy scores (score, score2, score3), and their predictions for three specific input scenarios (Prediction-1, Prediction-2, and Prediction-3). This DataFrame is then displayed using comparison_df.head().

CONCLUSION

This analysis used machine learning models, including Linear Regression, Decision Tree, and Random Forest, to predict student performance based on factors like study hours, extracurricular activities, sleep, and practice. Random Forest delivered the best accuracy, highlighting the potential of ML in forecasting academic outcomes and guiding student success improvements.

THANK YOU!!!